D2 - Secure Symmetric Encryption Module

Diseño del cifrado

Algoritmo de AEAD

Para el módulo de cifrado se escogió el algoritmo de cifrado de bloques AES-256-GCM que proporciona la biblioteca de cryptography, porque es un algoritmo estandarizado por parte del NIST y cuenta con optimización a nivel de hardware en CPUs modernos, el modo de operación con GCM le añade el modo contador (cifrado) y galois (autenticación) que cumple el requerimiento AAD.

Esta implementación genera una llave para cada archivo cifrado, esta llave de archivo se almacena cifrada en el contenedor del archivo correspondiente, en tiempo de ejecución se realiza un proceso de derivación de llaves a partir de una contraseña de usuario para descifrar las llaves de archivo y posteriormente el contenido de los archivos.

Tamaño de llave

Para la implementación de AES-GCM se utiliza un tamaño de llave 256 bits, el cual es el tamaño máximo aceptado en el estándar, para la derivación de la llave maestra a partir de la contraseña de usuario se utilizó un algoritmo de hash SHA256 con una llave de 256 bits, estos tamaños se alinean con las recomendaciones del NIST para el cifrado simétrico.

Estrategia de nonce

Para el nonce, se genera un numero de 96 bits generado al azar, para cada archivo cifrado con el programa se genera un nonce único utilizando la biblioteca secrets, la cual es un generador de números pseudoaleatorios criptográficamente seguro y que utiliza recursos de hardware para su generación. cada archivo se almacena en directorio junto con los datos necesarios para descifrarlo , generar un nonce por cada archivo cifrado reduce enormemente el riesgo que ocurra la repetición de nonce y mantiene las propiedades de confidencialidad de la información.

Autentificación de metadatos

Para la autentificación de metadatos se almacena junto al texto cifrado un header que contiene los metadatos del archivo, estos se utilizan para especificar información relevante del archivo y parámetros del descifrado tales como el tamaño de la llave, el algoritmo usado, el tamaño del nonce, etc, estos metadatos se adjuntan como información autenticada adicional (AAD) para garantizar la integridad y detectar modificaciones de los datos.

Decisiones de seguridad

- ¿Por qué utilizar AEAD en lugar de un esquema de cifrado + hash?

El uso de un sistema de cifrado autentificado con datos asociados (AEAD) proporciona un mecanismo de confidencialidad e integridad dentro del mismo proceso de cifrado, el algoritmo, en este caso AES-GCM, simultáneamente hace el cifrado del texto plano y genera una etiqueta de autentificación que se concatena al

texto cifrado, el uso de esta etiqueta de autentificación permite garantizar la integridad del texto cifrado y de los metadatos asociados detectando cualquier modificación durante el proceso de descifrado.

Resulta más conveniente este sistema ya que a diferencia de un sistema de cifrado + hash simple es mucho menos susceptible a errores causados por una mala implementación además de reducir los problemas causados por metadatos faltantes mientras simultáneamente garantiza la integridad de estos metadatos. En general podemos decir que el uso de AEAD presenta una solución más robusta y estandarizada para el proceso de autentificación y verificación de integridad de los datos.

- ¿Qué sucede si se repite un nonce?

Esto representa un fallo crítico para la seguridad del algoritmo AES-GCM, si se repite un nonce junto con una llave privada el flujo de bits generados por el algoritmo se repetirá, de esta manera un atacante podría aplicar una operación XOR a ambos textos cifrados para obtener una parte del texto original.

Además de esto la reutilización de un nonce y una llave privada permite a un atacante recuperar la llave de autentificación, comprometiendo la autentificación de toda la información cifrada con esa llave, de esta manera un atacante podría acceder a la información confidencial además de alterarla sin ser detectado.

- ¿Contra qué tipo de atacante está protegiendo este esquema?

El sistema está diseñado para proteger contra un atacante que puede acceder a y manipular los archivos cifrados almacenados en la bóveda, se considera que un atacante podría intentar leer y alterar el texto cifrado, alterar los metadatos o reemplazar partes del contenedor para burlar los mecanismos de autentificación, confidencialidad e integridad.

Esta implementación asume que el atacante tiene acceso a los archivos de documentos cifrados, pero no conoce la contraseña del usuario o la llave derivada de esta.

Pruebas de Seguridad

1. Prueba de autenticación

El programa desarrollado solicita una contraseña para cifrar el archivo y posteriormente requiere la misma contraseña para realizar el descifrado del archivo ya cifrado.

Para esta prueba se eligió una contraseña específica durante el proceso de cifrado. Sin embargo, al intentar descifrar el archivo se ingresó una contraseña diferente.

Como resultado, el sistema generó un bloqueo. Esto ocurrió porque, al intentar descifrar el archivo utilizando una clave incorrecta, el algoritmo AES-GCM detectó que los datos no eran auténticos.

Esto sucede porque, al usar una clave incorrecta, el código de autenticación resultante (TAG) no coincide con el TAG esperado, lo que indica que la autenticación del mensaje ha fallado y evita que se genere cualquier

salida de texto plano.

```
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ vim prueba1.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt prueba1.txt
Vault container =  vault_container/prueba1.txt
Enter your password:
Container encryption successful, created at:  vault_container/prueba1.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py decrypt vault_container/prueba1.txt salida.txt
Enter your password:
ERROR: Authentication failed: password may be incorrect or the encrypted key may have been tampered with
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$
```

2. Prueba de inyección de errores

Para esta prueba se realizó la inyección de bits basura en un archivo cifrado. En este caso, el archivo hola.txt contenía información sensible: "Información súper secreta".

Después de cifrarlo correctamente, se utilizó el comando dd para sobrescribir el offset 10 del archivo ciphertext con el valor hexadecimal \xff.

Una vez realizada esta modificación, al intentar ejecutar el proceso de descifrado se generó una excepción de tipo cryptography.exceptions.InvalidTag. Esto significa que, durante el descifrado, el algoritmo recalculó el código de autenticación y detectó que no coincidía con el TAG original almacenado al final del archivo.

Como resultado, el sistema detuvo el proceso de descifrado de manera inmediata, antes de producir cualquier salida de texto plano. Esto confirma que el mecanismo de autenticación e integridad del cifrado detecta correctamente modificaciones maliciosas en el archivo cifrado.

```
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ vim hola.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ cat hola.txt
Informacion super secreta
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt hola.txt
Vault container =  vault_container/hola.txt
Enter your password:
Container encryption successful, created at:  vault_container/hola.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ printf '\xff' | dd of=vault_container/hola.txt/ciphertext bs=1 seek=10 count=1 conv=notrunc
1+0 records in
1+0 records out
1 byte copied, 7.1963e-05 s, 13.9 kB/s
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py decrypt vault_container/hola.txt o.txt
Enter your password:
ERROR: Authentication failed: password may be incorrect or the encrypted key may have been tampered with
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$
```

3. Prueba de modificación de metadatos

En esta prueba se realizaron cambios en el archivo header.json. Este archivo contiene la información de los metadatos del archivo cifrado. La modificación se efectuó antes de intentar el proceso de descifrado.

Durante el descifrado, el algoritmo utiliza el contenido del archivo JSON como datos asociados para verificar el TAG de autenticación. Sin embargo, debido a que el archivo header.json ya no es idéntico al que existía en el momento del cifrado, el cálculo del TAG no coincide con el valor original.

Como consecuencia, la librería detecta que los datos asociados han sido modificados. Esto impide que el proceso de descifrado continúe y protege la integridad de la información.

```
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ vim metadato.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt metadato.txt
Vault container =  vault_container/metadato.txt
Enter your password:
Container encryption successful, created at:  vault_container/metadato.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ nano vault_container/metadato.txt/header.json
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py decrypt vault_container/metadato.txt try.txt
Enter your password:
ERROR: Authentication failed: password may be incorrect or the encrypted key may have been tampered with
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$
```

PARA LAS SIGUIENTES PRUEBAS SE IMPRIMIERON ALGUNAS VARIABLES DE DEPURACION PARA MOSTRAR MEJOR EL COMPORTAMIENTO

4. Prueba de consistencia del cifrado y descifrado

Esta prueba tiene como objetivo verificar que no se pierda ni se modifique información durante los procesos de cifrado y descifrado. Se espera que el contenido del archivo original sea exactamente el mismo que el obtenido después de descifrar el archivo previamente cifrado.

Para realizar la prueba se utilizó el archivo Ciframe.txt, cuyo contenido era "HOLA DESCÍFRAME". Posteriormente, se cifró el archivo y luego se realizó el proceso de descifrado. El resultado del descifrado se guardó en un archivo llamado salida.txt.

Al revisar el contenido del archivo salida.txt, se observó que la información coincide completamente con el contenido original, lo que confirma que el proceso de cifrado y descifrado no produjo modificaciones ni

pérdida de datos.

```
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ vim ciframe.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ cat ciframe.txt
HOLA CIFRAME
`

marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt ciframe.txt
Namespace(command='encrypt', input_file='ciframe.txt')
Enter your password:
Master key is:  c6fc3fb37db3ffb34cd619be006cb5058754c5b1a70111719fb456638e0a4594
Vault container =  vault_container/ciframe.txt
New key generated, key is:
 7f8ce114b7773e72370d5481bddf7c35fe98401ec494277481745a14693fb300
CT=  72d21b574546cb0da1dc0c672a30d705fb6f1d84327dbc38c93e4812cda4413d
TAG=  fb6f1d84327dbc38c93e4812cda4413d
Encrypted key is:
 eebd5c26d6a98ce1ed83e2c290005cb2f0a031e5b6b040662a73471c4cfac1ac6bf8b2aa7283d893bd0d0aecf33013a8
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py decrypt vault_container/ciframe.txt salida.txt
Namespace(command='decrypt', container_dir='vault_container/ciframe.txt', output_file='salida.txt')
Enter your password:
Master key is:  c6fc3fb37db3ffb34cd619be006cb5058754c5b1a70111719fb456638e0a4594
Container decryption successful, contents written to:  salida.txt
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ cat salida.txt
HOLA CIFRAME
`
```

5. Prueba de Cifrados diferentes

Aunque se cifre el mismo archivo 100 veces, no debería generarse la misma salida de cifrado en cada ocasión. Esto se debe al uso de un nonce, que modifica el estado inicial del algoritmo y garantiza que el texto cifrado resultante sea diferente en cada ejecución.

De esta manera se evita la aparición de patrones en los datos cifrados, lo que fortalece la seguridad del sistema. Además, los algoritmos criptográficos modernos presentan el llamado efecto avalancha, lo que significa que si se modifica incluso un solo bit en la entrada, más de la mitad de los bits del resultado final cambian, produciendo un cifrado completamente distinto.

```
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt hola.txt
Namespace(command='encrypt', input_file='hola.txt')
Enter your password:
Master key is:  08749bb923d08c7d943f9708e49cdcb41b9fc707839e5234607b35003ebb9114
Vault container =  vault_container/hola.txt
New key generated, key is:
 0d1d020ee07e3d4b5577d408403bfa39dcc08e3f67585ddcd3f1020b185f25c7
CT=  a9758d1ba0859f82dde6e4deedbbe702e9156ab57c548cbe1fb6ddb0369fe499a264d6fac1d7fbb93032
TAG=  ddb0369fe499a264d6fac1d7fbb93032
Encrypted key is:
 b7f312a9626bda7b215b46ffc3d8e2da876e4cafba156f7a5742197e73529c9b914202fb97f2dc2b65c173c5afd5ce7c
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt hola.txt
Namespace(command='encrypt', input_file='hola.txt')
Enter your password:
Master key is:  08749bb923d08c7d943f9708e49cdcb41b9fc707839e5234607b35003ebb9114
Vault container =  vault_container/hola.txt
New key generated, key is:
 d4d6c40ab76e18da229be2c7892c0a2a93a46587f1172a58c2b7b26ec7e80fb6
CT=  0ec747e656b791d4e5e650b70f2fe2781d0b730f2fddbddb1ce8121fd5de984c5b42fac66d7acf19ba1f
TAG=  121fd5de984c5b42fac66d7acf19ba1f
Encrypted key is:
 ee4063ddd9a5c4ddcc4224150456b00e959fb270c2cdc1d120975dfed1ef401a8a663ef9c286ee6c7d45de8d80b989e3
marialu@Ubuntuchis:~/criptografia/criptografia2026-2/src$ python3 main.py encrypt hola.txt
Namespace(command='encrypt', input_file='hola.txt')
Enter your password:
Master key is:  08749bb923d08c7d943f9708e49cdcb41b9fc707839e5234607b35003ebb9114
Vault container =  vault_container/hola.txt
New key generated, key is:
 56b88629b2cc96e55e65e4e9a4a6429756962bd08f1e21c0a067f35b802c54c6
CT=  f9b18106f05b41a5e4e89012ea7d45504baea8a13778ce5fe2f67c2503c590bdd5399a3eae5d3c40cdf2
TAG=  7c2503c590bdd5399a3eae5d3c40cdf2
Encrypted key is:
 d1f29d564c27c694612d1f5fe82a986c1ad3a60ae5c6e90a516fb08177e1d3f8f3708b606e32b90a26bee27f0141a0b2
```